Red Hat AI (RHAI) 3.5

New Key Features

Version 1, 2026-08-25

Home

Introduction

Course Title: Red Hat AI (RHAI) 3.5: New Key Features

Description: Welcome to the Red Hat AI (RHAI) 3.5 Enterprise Technical Training Course. This training guide covers the key architectural and functional updates in version 3.5.

This course is designed around the four core product pillars that drive the RHAI 3.5 strategy:

- Fast & Efficient Inference
- Connecting Data to Models
- Agentic AI
- Platform Infrastructure, Scale & Security

Each module represents a product pillar. Within each module, lessons are structured around the official Key Messages, with technical explanations, operational mechanics, and enterprise use-case scenarios.

Duration: ...

Technology Preview Features

Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process. For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scop.

Developer Preview Features

Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to functionality in advance of possible inclusion in a Red Hat product offering. Customers can use these features to test functionality and provide feedback during the development process. Developer Preview features might not have any documentation, are subject to change or removal at any time, and have received limited testing. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA. For more information about the support scope of Red Hat Developer Preview features, see Developer Preview Support Scope.

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Sarvesh Pandit (Principal Technical Training Developer)
- Carlos Condado (Senior Product Marketing Manager, AI)
- Will McGrath (Senior Principal Product Marketing Manager, AI)
- Martin Isaksson (Principal Architect, AI)
- Younes Ben Brahim (Principal Product Marketing Manager, AI)
- Rutuja Deshmukh (Technical Training Developer)
- Anna Swicegood (Learning Product Manager)

Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Fast & Efficient Inference

Production-ready agentic AI requires inference capabilities that go beyond simple text completion. Red Hat AI 3.5 gives you the operational depth, model safety transparency, resource governance, and multimodal foundations to deploy and manage high-priority LLM workloads at scale.

This section focuses on:

- Model Safety and Reliability Evidence Before Deployments
- The Operational Controls Shared Inference Requires
- The Foundation for AI Agents that Work Beyond Text

Model Safety and Reliability Evidence Before Deployments

Enterprise teams in regulated industries need to verify that a model behaves safely and reliably under adversarial conditions before promoting it to production.

Garak Security & Safety Benchmarking Integration [GA]

The Governence Rule

Verify that a model behaves safely and reliably under adversarial conditions before promoting it to production.

Persona Tension

SRE Lead: How can we verify that a model behaves safely under adversarial conditions before promoting it to production in a regulated environment?

Platform Engineer: We integrate Garak benchmark results natively into the Model Catalog under the Safety & Security Insights tab to display a detailed risk profile for every validated model.

Think First: Question

Before examining the mechanics below: How does the Model Catalog display security risk profiles without requiring manual red-teaming for every imported model?

The Core Answer

The automated validation pipeline executes pre-configured Garak scans via the EvalHub REST API, ingesting structured JSON payloads directly into the Model Catalog UI.

What is it?

Garak (Generative AI Red-teaming & Assessment Kit) is an open-source security scanning framework specifically designed to probe LLMs for vulnerabilities. In RHAI 3.5, Garak benchmark results are natively integrated into the platform's Model Catalog under the Safety & Security

Insights tab, providing a detailed risk profile for every validated model.

How it works?

During the automated model validation pipeline, the model validation team triggers pre-configured security evaluation collections (red teaming, prompt injection, jailbreaking, and data privacy/PII leakage scans) via the EvalHub REST API. These scans run adversarial prompts against the model endpoint. The resulting structured JSON payloads are ingested by the Model Catalog's BFF (Backend-for-Frontend) and displayed in the UI as an Evaluation benchmarks table. This table includes columns for evaluation name, category, benchmark, description, and numeric result score.

Enterprise Use Case Scenario

A financial services institution is deploying an on-premise, air-gapped LLM to assist with retail banking customer queries. Due to strict compliance requirements under the EU AI Act and NIST AI RMF, they cannot pull models from public repositories without proof of safety. Using the Model Catalog's Safety & Security Insights tab, platform administrators can immediately verify the model's resistance to jailbreaking and prompt injection, bypassing weeks of manual red-teaming and legal approvals.

Tool-Calling Validation [GA]

The Developer's Challenge

What prevents our API calls from failing silently when models generate malformed arguments or invalid JSON schemas during execution?

Architectural Dimension	Traditional Validation	RHAI 3.5 Innovation
Tool Execution	Custom client-side parsing code required to handle malformed arguments.	Tool-Calling Validation: Confirmed models display validated vllm serve arguments directly in the catalog.

Think First: Question

Before examining the mechanics below: How do developers ensure model tool invocations conform to expected schemas without writing custom client-side parsing code?

The Core Answer

By selecting a model labeled with the Tool calling task filter in the RHAI Catalog, the development team starts with a pre-configured template, ensuring that tool invocations conform to the expected schema without custom client-side parsing code.

What is it?

Tool-Calling Validation is a quality verification layer in RHAI 3.5 that guarantees selected models are capable of reliably executing tool calls (such as API invocations and database queries) under standard enterprise conditions.

How it works?

An initial set of high-demand models is validated by the Red Hat Model Validation team specifically for tool-calling capabilities. This validation tests whether a model correctly structures invocations, accurately passes arguments, and correctly handles tool response schemas. Models with confirmed support display a Validated Arguments section on their Model Details page in the Model Catalog. Users can expand the Tool Calling panel to view and copy the exact vllm serve CLI arguments (e.g., --tool-call-parser, --chat-template, --enable-auto-tool-choice) required for correct deployment.

Enterprise Use Case Scenario

A logistics company builds a customer service agent that needs to invoke a proprietary parcel-tracking API. If the model generates malformed arguments or invalid JSON schemas, the API call fails silently. By selecting a model labeled with the Tool calling task filter in the RHAI Catalog, the development team starts with a pre-configured template, ensuring that tool invocations conform to the expected schema without custom client-side parsing code.

The Operational Controls Shared Inference Requires

Enterprise teams run agentic pipelines, batch processing jobs, and latency-sensitive API calls on shared infrastructure. RHAI 3.5 gives you the operational controls to keep competing workloads from disrupting one another.

SLO-Aware Serving [GA]

Architectural Dimension	Traditional Serving	RHAI 3.5 Innovation
Traffic Prioritization	Single tenant monopolization; batch queues starve live interactive calls.	SLO-Aware Serving: Endpoint Picker (EPP) tracks queues/KV cache to prioritize live traffic within TTFT thresholds.

Persona Tension

SRE Lead: How do we keep competing workloads like live chat and background batch processing from disrupting one another on shared GPUs?

Platform Engineer: We deploy SLO-Aware Serving in the llm-d inference engine to dynamically prioritize interactive traffic over background queues.

Think First: Question

Before examining the mechanics below: How does admission control guarantee latency targets for interactive requests without starving asynchronous batch queues?

The Core Answer

The Endpoint Picker tracks request queues and KV cache utilization to route high-priority traffic ahead of low-priority workloads.

What is it?

SLO-Aware Serving is an admission control and traffic shaping mechanism in the llm-d distributed inference engine that guarantees high-priority, latency-sensitive requests meet their Service Level Objectives (SLOs) when sharing GPUs with lower-priority workloads.

How it works?

he system introduces admission control, fairness policies, and starvation protection. The Endpoint Picker (EPP) tracks request queues and KV cache utilization. High-priority interactive requests are routed ahead of batch or low-priority workloads. It prevents any single tenant from monopolizing shared GPU resources, ensuring latency-sensitive requests (e.g., live chatbot responses) are served within their target Time-to-First-Token (TTFT) threshold while gracefully delaying asynchronous batch queues without starving them.

Enterprise Use Case Scenario

A telecommunications provider (such as Telenor) runs a shared GPU cluster serving both an interactive customer chat system (demanding low latency) and a nightly batch job summarizing customer service transcripts. During peak hours, interactive customer traffic surges. SLO-aware serving dynamically prioritizes the live chat requests over the batch processing, maintaining the SLA of the consumer chat application without needing to over-provision dedicated GPU pools.

Controlled Deployments (Canary and Rolling Updates) [GA]

Persona Tension

SRE Lead: How do we prevent production downtime or performance degradation when transitioning between model versions?

Platform Engineer: Controlled Deployments allow us to safely test updates with live canary traffic and trigger automated rollbacks if performance metrics degrade.

Think First: Question

Before examining the mechanics below: What mechanism protects the end-user experience when a latency anomaly occurs during a model upgrade?

The Core Answer

The deployment engine monitors live performance metrics and automatically triggers a rollback to the previous stable engine upon detecting anomalies.

What is it?

Controlled Deployments is a model lifecycle management feature that allows platform teams to safely deploy, test, and transition between model versions in production with zero downtime and automated rollback.

How it works?

The platform supports rolling updates, canary releases, and inference-metric-based rollback. When upgrading a model, administrators can route a small fraction (e.g., 5%) of live production traffic to the new version. The system monitors live performance metrics (request success, latency, error rates). If performance degrades below a specified threshold, the system automatically triggers an automated rollback, reverting traffic to the previous version. In-flight requests on the newer model are allowed to complete gracefully before the pods are decommissioned.

Enterprise Use Case Scenario

An e-commerce retailer is upgrading its product recommendation LLM from version 1.0 to 1.1. During the transition, a sudden spike in latency occurs on the new version due to an un-optimized serving parameter. RHAI's controlled deployment engine detects the latency anomaly and automatically rolls back all traffic to the stable v1.0 engine, protecting the customer checkout experience from disruption.

Tool Calling Through Disaggregated Serving Path [GA]

Architectural Dimension	Traditional Serving	RHAI 3.5 Innovation
Execution Topology	Combined prefill and decode execution; scaling breaks stateful connections.	Disaggregated Serving: Prefill and decode split onto dedicated pools while preserving tool-use structures.

Think First: Question

Before examining the mechanics below: Why is disaggregated serving economically advantageous for agents processing massive prompts with brief outputs?

The Core Answer

By deploying on a disaggregated prefill/decode architecture, the firm scales prefill and decode nodes independently, maintaining real-time tool execution speeds while reducing overall GPU infrastructure costs by 40%.

What is it?

Disaggregated serving splits model execution into separate prefill (prompt processing) and decode (token generation) phases on dedicated GPU pools. In RHAI 3.5, Tool Calling is fully supported through this disaggregated architecture, so teams no longer have to choose between infrastructure scale and agent capability.

How it works?

Upstream prefill nodes ingest the initial prompt and tool-use schemas, while downstream decode nodes generate the tool call tokens. Previously, distributed cluster routing broke the stateful connection required for schema-compliant tool calls. RHAI 3.5 ensures that the disaggregated serving path correctly preserves and completes tool-use structures across node boundaries, allowing developers to scale their model-serving layer across multiple compute nodes without

breaking agent capability.

Enterprise Use Case Scenario

An insurance firm deploys a complex claim processing agent that must query multiple SQL databases and process long claim PDFs. The prompt sizes are massive (requiring high prefill compute), but the final tool call is concise (low decode compute). By deploying on a disaggregated prefill/decode architecture, the firm scales prefill and decode nodes independently, maintaining real-time tool execution speeds while reducing overall GPU infrastructure costs by 40%.

KV Cache CPU and NVMe Offloading [GA / DP]

Architectural Dimension	Traditional Memory Limits	RHAI 3.5 Innovation
Context Memory	GPU VRAM limits force context truncation or cause Out-of-Memory crashes.	Cache Offloading: Dynamic shift of KV cache to CPU DRAM (GA) and NVMe storage (DP).

Persona Tension

SRE Lead: How do we prevent Out-of-Memory (OOM) crashes when models analyze massive documents or long multi-turn conversations?

Platform Engineer: KV Cache Offloading dynamically shifts cache segments from GPU VRAM to system CPU DRAM and NVMe storage, removing the memory ceiling.

Think First: Question

Before examining the mechanics below: How does offloading KV cache eliminate the need to truncate context in long-document processing?

The Core Answer

The engine swaps inactive context blocks to CPU or NVMe storage, retrieving them back into VRAM seamlessly when requested by the user.

What is it?

KV Cache Offloading is a memory optimization capability in the vLLM engine that allows the Key-Value (KV) cache to be shifted from high-cost GPU VRAM to host system memory (CPU DRAM) and fast storage (NVMe), eliminating the "context wall" that limits conversation length.

How it works?

In RHAI 3.5, KV Cache CPU offloading is Generally Available (GA), while NVMe storage offload is in Developer Preview (DP). When an active conversation or long document ingestion exceeds the physical limits of GPU VRAM, the engine dynamically transfers older or inactive segments of the KV cache to CPU memory, or further down to a POSIX NVMe filesystem tier. When the user returns to those context blocks, the system swaps them back into VRAM. This process removes the memory ceiling that typically forces agents to truncate or summarize long conversations.

Enterprise Use Case Scenario

A legal tech corporation runs an AI document-review assistant that analyzes 500-page contract files. Normally, loading the entire context of several documents simultaneously would cause the model's pod to crash with an Out-of-Memory (OOM) error or force context truncation (degrading review accuracy). With CPU and NVMe cache offloading, the assistant retains the entire context of the legal document across multi-turn queries, ensuring high-fidelity reasoning over the complete file.

The Foundation for AI Agents that Work Beyond Text

Enterprise AI use cases are evolving to require multimodal models that process images, audio, and documents alongside text. Servicing these needs on separate infrastructure silos adds operational complexity you don't need.

Unified Multimodal Inference (vLLM Omni) [DP]

Think First: Question

Before examining the mechanics below: What is the primary operational benefit of serving vision, text, and audio models under one container configuration?

The Core Answer

It reduces configuration overhead and eliminates cross-network latency between single-modality models.

What is it?

vLLM Omni (Developer Preview) is an extension to the RHAI model serving layer that serves multiple input/output modalities—specifically text, audio, and images—within a single, unified serving configuration.

How it works?

Instead of routing multimodal requests across separate model pipelines and infrastructure configurations, vLLM Omni hosts models capable of processing multiple modalities (e.g., vision-language models like Granite Vision) under a single KServe deployment. A request containing an image and a text prompt is received, processed, and responded to through the same container configuration, reusing the same physical GPU allocation.

Enterprise Use Case Scenario

An automotive manufacturer deploys an AI technician assistant on the factory floor. The assistant receives high-resolution photos of engine parts and voice clips describing mechanical symptoms, and responds with a text-based repair guide. With vLLM Omni, the manufacturer runs the entire application through a single, unified serving infrastructure on RHAI, reducing configuration overhead and eliminating cross-network latency between single-modality models.

Connecting Data to Models

Raw foundation models are functionally limited without access to enterprise data. RHAI 3.5 closes this gap with production-grade data integration pipelines, advanced model-tuning frameworks, and robust evaluation systems to build trustworthy AI applications.

This section focuses on:

- AutoRAG and AutoML: From Enterprise Data to Production AI
- Post-Training and Inference-Time Scaling: Improve Model Reasoning
- AI Evaluations and AI Safety: Build Confidence Before Production

AutoRAG and AutoML: From Enterprise Data to Production AI

Deploying data-intensive AI patterns requires transitioning unstructured documents and tabular datasets into production-ready endpoints with less manual work.

AutoRAG Platform [TP]

Think First: Question

How does AutoRAG automate the evaluation and determination of the optimal Retrieval-Augmented Generation (RAG) pipeline configuration for a specific dataset and use case?

The Core Answer

AutoRAG introduces a suite of enterprise features including Multilingual AutoRAG (natively processing German, Spanish, and Japanese documents), Contextual Retrieval (adding brief context summaries during chunking to prevent semantic loss), Deploy RAG Pattern as an Endpoint (deploying a validated configuration as a standard API endpoint with a single click), Visual Pipeline Debugging (providing step-by-step execution traces), and Natural Language Chatbot Testing.

What is it?

AutoRAG is an automated optimization platform built into the RHAI dashboard that evaluates and determines the optimal Retrieval-Augmented Generation (RAG) pipeline configuration for a specific dataset and use case.

How it works?

AutoRAG 3.5 introduces a suite of enterprise features:

- **Multilingual AutoRAG**: Natively processes German, Spanish, and Japanese documents without upfront translation.
- **Contextual Retrieval**: Automatically adds brief context summaries to individual document chunks during chunking, preventing semantic loss.

- **Deploy RAG Pattern as an Endpoint**: Allows users to deploy a validated RAG configuration as a standard API endpoint (accessible via curl, Go, or Python) with a single click.
- **Visual Pipeline Debugging**: Provides step-by-step execution traces to pinpoint where a RAG pipeline fails.
- **Natural Language Chatbot Testing**: Allows conversational testing of the selected pattern before deployment.

Enterprise Use Case Scenario

A global pharmaceutical giant has multi-lingual clinical trial documents in Spanish, Japanese, and English. They need a RAG system to answer research queries. Instead of manually tuning chunk sizes, embedding models, and vector stores, they run AutoRAG. AutoRAG evaluates hundreds of combinations, applies Contextual Retrieval to ensure fragments retain meaning, and exports a production-ready API endpoint—saving months of data engineering effort.

AutoML AutoGluon Serving Runtime in KServe [TP]

Persona Tension

SRE Lead: Why should we write custom serving containers to deploy traditional predictive machine learning models trained on tabular data?

Platform Engineer: We don't need to. KServe natively integrates the AutoGluon ServingRuntime to expose prediction endpoints directly from the AI hub leaderboard.

Think First: Question

Before examining the mechanics below: How does native AutoGluon integration in KServe streamline model deployment?

The Core Answer

It allows top-ranked tabular models to be registered and served via standard inference APIs using modelFormat: autogluon without custom containers.

What is it?

This feature integrates AutoGluon (an automated machine learning toolkit for tabular data) as a native serving runtime in KServe, closing the train-to-serve loop for traditional predictive AI.

How it works?

RHAI AutoML allows users to train and rank machine learning models on a leaderboard. Once the top model is selected, administrators can register it in the AI hub and deploy it natively with modelFormat: autogluon. KServe provisions the AutoGluon ServingRuntime, allowing applications to score predictions via standard KServe v1/v2 inference APIs without custom containers.

Enterprise Use Case Scenario

A retail bank uses AutoML to train a customer churn prediction model on historical transactional

data. Previously, deploying the resulting AutoGluon model required writing a custom Flask container. With the new native runtime, the data science team promotes the churn model from the leaderboard directly to KServe, exposing a production-ready prediction endpoint in minutes.

Kubeflow Spark Operator [TP]

The Data Processing Rule

Orchestrate large-scale distributed Spark jobs directly from notebooks while managing pod scheduling and network policies natively.

Think First: Question

How does the Kubeflow Spark Operator integrate with RHAI workbenches to unify data preparation with subsequent training and inference workflows on a single platform?

The Core Answer

The operator is integrated directly with RHAI workbenches, allowing users to define, run, and manage distributed Spark workloads using SparkApplication custom resources from within their existing notebook environment for interactive, real-time job management.

What is it?

The Kubeflow Spark Operator (Technology Preview) integrates Apache Spark with RHAI, so users can orchestrate large-scale, distributed data-processing and preparation jobs directly from their workbench.

How it works?

The operator is integrated directly with RHAI workbenches. Users can define, run, and manage distributed Spark workloads (using SparkApplication custom resources) from within their existing notebook environment. This integration enables interactive, real-time job management and unifies data preparation with subsequent training and inference workflows on a single platform.

Enterprise Use Case Scenario

A smart-grid utility company processes gigabytes of hourly IoT sensor readings before feeding them into a forecasting model. Using the Kubeflow Spark Operator integration, a data engineer triggers a distributed Spark data-cleaning job directly from their Jupyter notebook workbench. RHAI handles the orchestration and monitoring of the Spark application on the cluster, preparing the dataset for training without the data scientist ever having to leave their workbench environment.

Post-Training and Inference-Time Scaling: Improve Model Reasoning

Post-training optimization aligns models with domain-specific formatting and safety expectations—while keeping inference costs in check.

Budget-Efficient Inference-Time Scaling [GA / DP]

The Cost Optimization Rule

Dynamically increase model reasoning depth on difficult queries while stopping early on simple ones to optimize GPU compute utilization.

Persona Tension

SRE Lead: How do we prevent complex reasoning queries from driving up GPU compute costs across simple, routine requests?

Platform Engineer: Inference-Time Scaling applies Adaptive Early Stopping to resolve simple queries in early evaluation batches while scaling depth for hard ones.

Think First: Question

How does Inference-Time Scaling (ITS) dynamically adjust model reasoning depth across simple and complex queries to optimize GPU compute utilization?

The Core Answer

Inference-Time Scaling introduces two self-consistency variants with Adaptive Early Stopping: Adaptive Self-Consistency (generating samples in exponentially growing batches like 2 → 4 → 8 and stopping as soon as a configurable supermajority threshold is met) and Beta Self-Consistency (firing requests concurrently and canceling outstanding ones the moment a Bayesian confidence threshold is reached).

What is it?

Inference-Time Scaling (ITS) is a budget-aware serving capability that dynamically increases model reasoning depth on difficult queries while stopping early on simple ones, optimizing GPU compute utilization.

How it works?

RHAI 3.5 introduces two self-consistency variants with Adaptive Early Stopping:

- **Adaptive Self-Consistency**: Generates samples in exponentially growing batches (e.g., 2 → 4 → 8) and stops as soon as a configurable supermajority threshold (e.g., 75% agreement) is met.
- **Beta Self-Consistency (Experimental)**: Fires requests concurrently and cancels outstanding ones the moment a Bayesian confidence threshold is reached.

Enterprise Use Case Scenario

A tax consulting firm deploys a chatbot to answer tax law questions. Simple queries (e.g., "What is the standard deduction?") are resolved in the first batch of 2 samples, saving inference cost. Complex queries (e.g., "How does depreciation apply to a specific corporate merger?") trigger the full evaluation budget of 8 samples, ensuring high-fidelity reasoning when it is genuinely needed.

AI Evaluations and AI Safety: Build Confidence Before Production

Before going to production, you need objective, auditable metrics that prove your AI system behaves accurately and meets corporate safety standards.

EvalHub [GA]

Think First: Question

How does EvalHub operate as a unified platform to scientifically validate the performance, safety, and adversarial resilience of models, RAG pipelines, and agentic workflows prior to real-world deployment?

The Core Answer

EvalHub provides key capabilities including Multi-Tenant Deployment (namespace-admin-managed per-tenant deployment with tenant-scoped collections), Kueue Integration (fair-share scheduling of massive evaluation workloads on LocalQueues), Pluggable Framework Adapters (adapters for RAGAS, DeepEval, Inspect AI, and SWE-Bench), Quality Gates & EvalCards (configurable pass/fail thresholds and automated EvalCard PDF generation), and Storage Independence (Persistent Volume Claims as storage sources for air-gapped environments).

What is it?

EvalHub is RHAI's unified platform for evaluating models, RAG pipelines, and agents. In version 3.5, EvalHub graduates to General Availability (GA) with a full set of production-ready features.

How it works?

EvalHub provides a unified interface to scientifically validate the performance, safety, and adversarial resilience of models, RAG pipelines, and agentic workflows prior to real-world deployment. Key capabilities include:

- **Multi-Tenant Deployment**: Namespace-admin-managed per-tenant deployment with tenant-scoped collections.
- **Kueue Integration**: Fair-share scheduling of massive evaluation workloads on LocalQueues.
- **Pluggable Framework Adapters**: First-class adapters for RAGAS, DeepEval, Inspect AI (UK Safety Institute), and SWE-Bench.
- **Quality Gates & EvalCards**: Configurable pass/fail thresholds in the UI and automated EvalCard generation (a structured PDF capturing model identity, configurations, and results for compliance).
- **Storage Independence**: Persistent Volume Claims (PVC) as storage sources for air-gapped environments.

Enterprise Use Case Scenario

A regulated healthcare insurer is deploying an automated claim approval agent. To satisfy corporate risk committees, they establish an automated evaluation pipeline in EvalHub. The system evaluates the agent on the Inspect AI safety framework and RAGAS (checking context recall). Upon completion, it generates an official, structured, versioned, and auditable EvalCard that is automatically archived for regulatory compliance audits under national healthcare frameworks.

AI Safety and Red Teaming Engine [GA / TP / DP]

The Gateway Security Rule

Deliver structured enterprise AI safety infrastructure enabling real-time agent-aware tool enforcement, simulated adversarial testing, and unified guardrail strategies before models and agents reach production.

Persona Tension

SRE Lead: How can we block malicious prompt injections and unauthorized tool executions without modifying underlying agent code?

Platform Engineer: We place NeMo Guardrails directly on the MCP gateway to intercept tool calls, alongside MiDojo red-teaming for adversary simulation.

Think First: Question

Before examining the mechanics below: How does RHAI translate complex regulatory requirements into concrete security test profiles?

The Core Answer

Unstructured Policy-to-Risk Mapping automatically maps policy text to over 600 risk identifiers in IBM's AI Risk Atlas Nexus, auto-generating Garak probe profiles.

What is it?

This capability delivers structured enterprise AI safety infrastructure, enabling real-time agent-aware tool enforcement, simulated adversarial testing, and unified guardrail strategies before models and agents reach production.

How it works?

RHAI 3.5 delivers safety across four key components:

- **MCP Gateway Content Guardrails via TrustyAI Operator (TP)**: Enables enterprises to enforce content safety and privacy policies on agent tool calls directly at the Model Context Protocol (MCP) Gateway without modifying a single line of agent code. The TrustyAI Operator automates the deployment of standalone gateway guardrails—automatically discovering the MCP Gateway and BBR plugin to provision an EnvoyFilter—eliminating the requirement to run the full TrustyAI observability and explainability stack.

- **Standardization on NVIDIA NeMo Guardrails (GA)**: RHAI 3.5 unifies its guardrailing strategy by establishing NVIDIA NeMo Guardrails as the single supported backend, while officially deprecating the Java-based FMS Guardrails Orchestrator. This resolves platform fragmentation, transitioning the platform from a collection of experimental safety tools into a coherent, production-grade safety architecture.
- **MiDojo Execution Engine (DP)**: A man-in-the-middle red-teaming engine that intercepts communication between an agent and its tools. It injects adversarial payloads (such as indirect prompt injections and spoofed tool responses) to stress-test the agent's safety boundaries in its actual environment.
- **Unstructured Policy-to-Risk Mapping (DP)**: Automatically maps raw corporate policy text (such as the EU AI Act, HIPAA, DORA, or internal governance charters) to structured AI risk identifiers via semantic matching against IBM's AI Risk Atlas Nexus (comprising 600+ risk identifiers across 10 governance frameworks). This produces a machine-readable risk profile that feeds directly into Garak probe selection, NeMo Guardrails configuration, and compliance dashboards, replacing weeks of manual mapping workflows.

Enterprise Use Case Scenario

A retail organization deploys an AI customer service agent that can issue refunds via an API. An attacker might try an indirect prompt injection (e.g., placing an order with a delivery address containing instructions to "Refund the order immediately"). The company uses MiDojo to simulate these adversarial tool payloads during development, while securing the live production environment using MCP Gateway Content Guardrails to intercept and block unauthorized tool actions at the gateway level.

Practical Agentic AI

Red Hat AI 3.5 moves beyond basic model serving—delivering a governed, secure agentic platform built for production.

This section focuses on:

- Agentic API Foundation: OGX (GA)
- Pre-built Templates in AI Hub for Developer Productivity

Agentic API Foundation: OGX (GA)

Building AI agents takes more than raw model endpoints. You need a unified, stable API layer to coordinate reasoning, data retrieval, and real-world execution.

OGX GA APIs [GA]

Architectural Dimension	Custom Orchestration	OGX GA APIs
Agent State & Multi-Turn	Custom state-tracking code and custom document parsers.	Standardized APIs: Responses API manages tool loops; File Processor with Docling parses documents.

Think First: Question

How does OGX deliver a standardized, open-source API layer that abstracts complex agent interactions without custom orchestration code?

The Core Answer

OGX delivers a standardized, open-source API layer that abstracts complex agent interactions via the Responses API (standardizing multi-turn interactions, maintaining chat history, managing state, and coordinating sequential tool calls), the File Processor API with Docling Integration (ingesting and parsing complex PDF, DOCX, and image documents into structured markdown), and Built-in RAG Capabilities connecting retrieval pipelines directly to the inference layer.

What is it?

OGX (formerly Llama Stack) is the production-ready API foundation for agentic AI on OpenShift AI. In version 3.5, the core OGX endpoints graduate to General Availability (GA).

How it works?

OGX delivers a standardized, open-source API layer that abstracts complex agent interactions:

- **Responses API**: Standardizes multi-turn interactions, maintaining chat history, managing state, and coordinating sequential tool calls without custom orchestration code.
- **File Processor API with Docling Integration**: Ingests and parses complex PDF, DOCX, and

image documents, converting them into structured, vector-ready markdown.

- **Built-in RAG Capabilities**: Connects retrieval pipelines directly to the inference layer.

Enterprise Use Case Scenario

A government agency is building a virtual advisor to answer public queries about tax filings. The advisor must hold a multi-turn conversation, ingest tax booklets (using the Docling-integrated File Processor API), and look up real-time tax brackets using database tools. With OGX, developers implement this complex reasoning loop in a single weekend by building on the standardized Responses API, knowing the deployment is fully supported under Red Hat's enterprise SLA.

Pre-built Templates in AI Hub for Developer Productivity

Building agents from scratch means manually assembling frameworks, tools, and configurations. Pre-configured templates speed this up.

AI Hub Agent Starter Kits [DP]

Think First: Question

How do Agent Templates and Starter Kits in the AI hub allow developers to deploy and customize reference implementations of common agentic patterns in minutes instead of manually assembling frameworks, tools, and configurations?

The Core Answer

The AI hub includes starter kits such as the OpenClaw Starter Kit (pre-configured with Kustomize manifests, native OpenTelemetry diagnostics-otel tracing into MLflow, and OpenShift RBAC integration) and the Claude Code Starter Kit (supporting direct Anthropic API connections or local model serving via vLLM and the OGX gateway).

What is it?

Agent Templates and Starter Kits in the AI hub are pre-configured, tested reference implementations of common agentic patterns that developers can deploy and customize in minutes.

How it works?

The AI hub includes starter kits such as:

- **OpenClaw Starter Kit (DP)**: An open-source, general-purpose agent pre-configured with Kustomize manifests, native OpenTelemetry diagnostics-otel tracing into MLflow, and OpenShift RBAC integration.
- **Claude Code Starter Kit (DP)**: A pre-configured environment for Anthropic's Claude Code agent, supporting direct Anthropic API connections or local model serving via vLLM and the OGX gateway.

Enterprise Use Case Scenario

An IT operations team wants to build an automated agent that can troubleshoot cluster alerts. Instead of researching frameworks and writing container files, they deploy the OpenClaw Starter Kit from the AI hub. The kit launches with a secure, sandboxed runtime, OTel tracing configured to MLflow, and immediate access to cluster-monitoring APIs, reducing development time from days to minutes.

Enterprise Platform Infrastructure, Scale & Security

Scaling AI operations across your organization takes solid platform governance. Red Hat OpenShift AI 3.5 unifies safety, observability, multi-tenancy, and hardware acceleration into a single operational view.

This section focuses on:

- AI Security: Automated Red Teaming and Security Insights
- Measurable Enterprise AI: Zero-Configuration Observability
- Multi-Tenancy for Enterprise AI
- GPU-as-a-Service (GPUaaS)

AI Security: Automated Red Teaming and Security Insights

Manual security validation creates a release bottleneck as AI model count grows. Automated safety orchestration removes this bottleneck.

"Safety & Security Insights" Tab in Model Catalog [GA]

Persona Tension

SRE Lead: How can internal data scientists evaluate model jailbreak resistance and prompt injection scores before choosing a model?

Platform Engineer: The Safety & Security Insights tab in the Model Catalog renders structured, color-coded evaluation results directly in the UI.

Think First: Question

How does the Safety & Security Insights tab display model security postures across different risk vectors directly in the RHAI AI hub UI?

The Core Answer

The UI uses unified Crimson team EvalHub UI table components to render a structured table (with columns for Evaluation name, Category, Benchmark, Description, and Result numeric score) automatically populated from the JSON output of automated Garak adversarial scans, featuring color-coded badges for visual distinction.

What is it?

The Safety & Security Insights tab is a GA extension of the RHAI AI hub that displays model security postures across different risk vectors directly in the UI.

How it works?

The UI uses the unified Crimson team EvalHub UI table components, rendering a structured table with columns for Evaluation name, Category (e.g., Safety Testing, Security Testing), Benchmark, Description, and Result (numeric score). The data is automatically populated from the JSON output of the automated Garak adversarial scans. Categories are visually distinguishable with color-coded badges, giving users clear visibility into risks like prompt injection and jailbreak resistance.

Enterprise Use Case Scenario

A data scientist needs to choose an LLM for an internal compliance chatbot. They browse the AI hub, open the model card, and select the Safety & Security Insights tab. They can quickly compare the jailbreak resistance scores of Granite vs Llama models, selecting the model that best satisfies internal corporate governance audits.

Measurable Enterprise AI: Zero-Configuration Observability

AI platforms need to be measurable. RHAI 3.5 ships with out-of-the-box observability dashboards, secure user-level access, and granular token-consumption tracking.

Centralized, Zero-Configuration Observability Stack [GA]

Think First: Question

On deployment, how does the operator bring the entire platform under a unified monitoring plane with zero manual configuration?

The Core Answer

On deployment, the operator automatically provisions and configures the Cluster Observability Operator (COO), OpenTelemetry (OTel), Tempo (distributed tracing), and Loki (structured log collection). Cluster administrators can turn on cluster-wide metrics and trace collection with zero manual configuration.

What is it?

This capability automates the day-0 lifecycle and deployment of RHAI's entire centralized observability stack.

How it works?

On deployment, the operator automatically provisions and configures the Cluster Observability Operator (COO), OpenTelemetry (OTel), Tempo (distributed tracing), and Loki(structured log collection). Cluster administrators can turn on cluster-wide metrics and trace collection with zero manual configuration, bringing the entire platform under a unified monitoring plane.

Enterprise Use Case Scenario

A platform operations team needs to deploy a monitoring stack across a new multi-node RHAI

cluster. Instead of manually installing and writing configurations for OTel collectors, Grafana datasources, and Prometheus rules, the operator deploys the complete, pre-configured observability stack automatically on Day-0, instantly ready to capture performance data.

Role-Based Metrics Proxy & Perses Dashboards [GA]

Persona Tension

SRE Lead: How can non-admin data scientists view latency and GPU metrics without granting them cluster-wide administrative access?

Platform Engineer: The Role-Based Metrics Proxy performs SubjectAccessReview checks and injects namespace label matchers into PromQL queries.

Think First: Question

How does RHAI unblock dashboard access for non-admin users to view performance metrics scoped strictly to their authorized projects without granting cluster-wide administrative access?

The Core Answer

RHAI uses a role-based proxy or Thanos Querier tenancy endpoints to intercept PromQL queries from the dashboard and perform standard Kubernetes SubjectAccessReview checks against the user's token, dynamically injecting namespace label matchers to ensure strict tenant isolation on Perses-based dashboards.

What is it?

This feature unblocks dashboard access for non-admin users, allowing data scientists and project owners to view performance metrics scoped strictly to their authorized projects.

How it works?

Previously, metrics dashboards were restricted to cluster-admins. In 3.5, RHAI uses a role-based proxy or Thanos Querier tenancy endpoints (supported by COO 1.5). The system intercepts PromQL queries from the dashboard and performs standard Kubernetes SubjectAccessReview checks against the user's token, dynamically injecting namespace label matchers (e.g., namespace="project-a"). This ensures strict tenant isolation on Perses-based dashboards, visualizing TTFT, throughput, and GPU utilization for authorized namespaces only.

Enterprise Use Case Scenario

A lead data scientist wants to monitor the latency percentiles and GPU memory utilization of their active customer-facing model. Instead of asking a cluster administrator to extract Prometheus metrics, they log into the RHAI dashboard. The role-based proxy verifies their project permissions and renders a Perses dashboard showing metrics exclusive to their namespace.

Structured Log Pipeline for MaaS Showback [TP]

Architectural Dimension	High-Cardinality Metrics	Loki Log Pipeline
Token Tracking	Prometheus metrics risk cardinality explosion on per-user tracking.	Gateway Log Pipeline: Loki tracks user, subscription, and model metadata for CSV export.

Think First: Question

How does the platform track and attribute token consumption on a per-user and per-subscription level without causing a high-cardinality metric explosion in Prometheus?

The Core Answer

The platform transitions from high-cardinality Prometheus metrics to a structured log pipeline at the gateway level, powered by Loki, tracking metadata (user, subscription, model, token counts) for every API request into structured Loki log databases and exposing CSV exports via the gen AI studio console.

What is it?

MaaS Showback (Technology Preview) is a structured log pipeline that tracks and attributes token consumption on a per-user and per-subscription level for billing and chargeback purposes.

How it works?

The platform transitions from high-cardinality Prometheus metrics to a structured log pipeline at the gateway level, powered by Loki. Every API request tracks metadata (user, subscription, model, token counts). The metrics flow into structured Loki log databases. Users and administrators can access dedicated dashboards in the gen AI studio console showing precise token consumption and rate-limit violations, which can be exported in CSV format.

Enterprise Use Case Scenario

An enterprise finance department allocates AI infrastructure costs back to individual business units (e.g., Marketing, HR). By checking the MaaS Showback dashboard, the platform operator exports a CSV showing that HR consumed 12 million tokens on a custom Granite model, while Marketing consumed 45 million tokens, so the finance department can run precise monthly billing.

Multi-Tenancy for Enterprise AI

Shared GPU clusters must be strictly partitioned to prevent tenants from accessing other groups' private data or monopolizing shared compute resources.

Complete Multi-Tenancy Isolation [GA]

The Isolation Rule

Auto-enforce isolation across Network Namespaces, Data/Secrets, and Persistent Storage out of the box.

Think First: Question

When a new tenant project is created, how does RHAI auto-enforce strict security boundaries across networking, data, and storage to prevent cross-namespace access?

The Core Answer

The system automatically enforces default isolation policies: Network Namespaces deny cross-tenant traffic using standard OpenShift NetworkPolicies, Data/Secrets Isolation ensures secrets and environment variables are strictly scoped to the tenant namespace, and Storage Isolation ensures persistent volumes remain completely inaccessible to other namespaces.

What is it?

RHAI 3.5's Multi-Tenancy Architecture is a layered security framework that provides strict isolation across namespaces, RBAC, networking, and data out of the box.

How it works?

When a new tenant project is created, the system auto-enforces isolation policies:

- **Network Namespace**: Denies cross-tenant traffic by default using standard OpenShift NetworkPolicies.
- **Data/Secrets Isolation**: Ensures secrets and environment variables are strictly scoped to the tenant namespace.
- **Storage Isolation**: Ensures persistent volumes are completely inaccessible to other namespaces.

Enterprise Use Case Scenario

A Cloud Service Provider (CSP) hosts an AI Factory serving multiple independent retail tenants on a single OpenShift cluster. If Tenant A is compromised or attempts to query Tenant B's namespace, the default network and storage policies block all cross-namespace communication, protecting proprietary model weights and customer datasets.

Dynamic Fair-Share GPU Scheduling [GA]

Architectural Dimension	Static GPU Quotas	Kueue Fair-Share Scheduling
Cluster Utilization	Idle GPUs remain locked to inactive teams (35% utilization).	Dynamic Borrowing: Idle GPUs lent to active teams; priority preemption reclaims capacity (70-85% utilization).

Persona Tension

SRE Lead: How can we maximize cluster GPU utilization without violating tenant capacity guarantees when demand spikes?

Platform Engineer: Kueue guarantees tenant quotas while dynamically lending idle GPUs,

executing priority preemption when the quota owner submits jobs.

Think First: Question

How does Kueue guarantee tenant GPU quotas while dynamically maximizing overall cluster utilization when tenant queues are idle?

The Core Answer

Each tenant is assigned a guaranteed GPU quota; if a tenant's queue is empty, the Kueue-native scheduler automatically lends their idle GPU capacity to accelerate other workloads, and the moment the quota owner submits a job, Kueue executes priority-based preemption to reclaim the borrowed resources within a configurable grace period, increasing average cluster utilization from 35% up to 70–85%.

What is it?

Built on upstream Kueue, this feature guarantees GPU quotas per tenant while dynamically lending idle compute resources to maximize overall cluster utilization.

How it works?

Each tenant is assigned a guaranteed GPU quota. If Tenant A's queue is empty, the Kueue-native scheduler automatically lends their idle GPU capacity to Tenant B to accelerate their workloads. However, the moment Tenant A submits a job, Kueue executes priority-based preemption, reclaiming the borrowed resources within a configurable grace period, guaranteeing Tenant A receives their allocated capacity. This dynamic sharing increases average cluster utilization from 35% up to 70–85%.

Enterprise Use Case Scenario

A supercomputing research center hosts a shared GPU cluster for various research groups. The Chemistry group is currently inactive, leaving their allocated GPUs idle. Kueue automatically lends these resources to the Physics group's large training run. When a Chemistry researcher submits a new forecasting job, the scheduler preempts the Physics workloads, returning the GPUs to the Chemistry queue.

GPU-as-a-Service (GPUaaS)

Platform administrators need clear visibility into hardware allocations and Kueue scheduling metrics—without piecing together command-line checks.

Integrated GPU Topology & Utilization Dashboard [GA]

The Platform Admin's Challenge

How can platform admins diagnose pending training jobs and inspect GPU hardware allocations without CLI commands?

Think First: Question

Before examining the mechanics below: How does the Infrastructure dashboard retrieve real-time metric data securely?

The Core Answer

The page runs entirely as a front-end host app within the odh-dashboard, querying metrics directly from Thanos via secure POST endpoints, with access restricted to cluster administrators.

What is it?

The Infrastructure page (GA) is an admin-only view built directly into the RHAI dashboard that provides real-time, consolidated visibility into GPU utilization, hardware inventories, and Kueue borrowing dynamics.

How it works?

The page is divided into 4 key visual sections:

- **Infrastructure Summary Cards**: Shows total accelerator count (allocated vs allocatable), compute utilization %, and memory utilization %.
- **Cohort Overview (Kueue-native)**: Displays utilization cards for each Kueue Cohort against the effective pool, alongside active admitted/pending workload counts.
- **Hardware Inventory**: A bar chart grouping active GPUs by specific hardware models.
- **Borrowing & Lending Trends**: A 7-day line chart tracking resource-sharing trends across ClusterQueues. The page runs entirely as a front-end host app within the odh-dashboard, querying metrics directly from Thanos via secure POST endpoints, with access restricted to cluster administrators.

Enterprise Use Case Scenario

A platform administrator receives a complaint from a data science team that their training jobs are stuck in a pending queue. Instead of running multiple terminal command-line checks, the admin opens the Infrastructure page. They instantly see that 100% of the cluster's NVIDIA H200 GPUs are allocated, and Kueue is currently executing a priority preemption to reclaim borrowed capacity, allowing them to provide an accurate estimate of when the jobs will resume.

Appendix

Resources

Download Course PDF

Red Hat AI 3 Documentation

Red Hat OpenShift AI Self-managed 3.5

Red Hat AI Inference 3.5

Red Hat Enterprise Linux AI 3.5